

Campus safety escort matching system design

 This is a copy of a chat between Claude and . Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. [Report](#)

hi, i need help creating a python file, the mission is The university is concerned about student safety on campus, especially during evening hours. Some students feel uneasy walking alone between buildings or across the campus. To address this, the university is planning a mobile application that connects students who need someone to accompany them with other students willing to help. Imagine a student leaving the library late at night who wants to walk safely to the dorms. They can submit a request through the app, indicating where they are, where they want to go, and the time they need help. On the other side, students volunteering as “helpers” can see requests and respond to them. Since many students may request assistance at similar times, the system needs to manage multiple incoming requests while ensuring that helpers are not overwhelmed or assigned in conflicting ways. In some situations, there may be no helpers available at all, while in others, several helpers may respond to the same request. The system must ensure that helpers receive only the information necessary to assist the student safely, without exposing sensitive details about the student. At the same time, helpers should be able to communicate with students quickly, safely, and reliably. The university expects that the system remains responsive even when many students are using it at the same time.

since i am only allowed to make one file, pls just do that. but do follow good structure and make classes

i know there is no previous data, but also make a change.dox file (make it clear what requirement you are fulfilling)

try to also cover edge cases, and have error handling. pls also leave comments in the code since python is new for us

also make it clear what section of the file is volunteers, student etc

in what way can we handle this? i guess a queue system. pls explain pro and cons between different ways and why you chose the one you do Since many students may request assistance at similar times, the system needs to manage multiple incoming requests while ensuring that helpers are not overwhelmed or assigned in conflicting ways.

perhaps before starting write me the requirements found from the text, and possible solutions to it, so we can decide together on how to build this system

Show less

I'll analyze the requirements and propose solutions before we start building. Let me break this down:

Requirements Analysis

Based on the text, here are the key requirements:

Functional Requirements:

1. **Student Request System:** Students can submit walk requests (location, destination, time)
2. **Helper Response System:** Volunteers can view and respond to requests
3. **Request Management:** Handle multiple simultaneous requests
4. **Helper Assignment:** Prevent conflicts, avoid overwhelming helpers
5. **Privacy Protection:** Share only necessary information with helpers
6. **Communication:** Quick, safe, reliable student-helper communication
7. **Availability Handling:** Manage scenarios with no helpers or multiple helpers

Non-Functional Requirements:

8. **Scalability:** System stays responsive under high load
9. **Reliability:** Ensure requests don't get lost or duplicated

Possible Solutions & Design Decisions

1. Request Queue Management

Options:

- **Priority Queue (FIFO):** First-come, first-served
 -  Pro: Fair, simple, predictable
 -  Con: Doesn't consider urgency or proximity
- **Priority Queue (weighted):** Consider time, distance, wait time
 -  Pro: More intelligent matching
 -  Con: More complex, could seem unfair
- **Broadcast System:** All helpers see all requests
 -  Pro: Fast matching, helper choice
 -  Con: Race conditions, overwhelming for helpers

My Recommendation: FIFO Queue with smart assignment - Simple, fair, and prevents helpers from being overwhelmed. We can add a "pending" state to prevent multiple helpers claiming the same request.

2. Helper Assignment Strategy

Options:

- **Manual Selection:** Helpers choose which requests to accept
 -  Pro: Helper autonomy
 -  Con: Popular requests get multiple responses, others ignored
- **Auto-Assignment:** System assigns closest available helper
 -  Pro: Efficient, prevents conflicts
 -  Con: Helpers lose choice
- **Hybrid (Offer-Accept):** System offers request to one helper at a time
 -  Pro: Prevents conflicts, gives helpers choice
 -  Con: Slower if helpers decline

My Recommendation: Hybrid Offer-Accept - Prevents conflicts while respecting helper autonomy. If a helper declines/doesn't respond in time, offer to the next available helper.

3. Concurrency & Thread Safety

Options:

- **Threading with Locks:** Use threading.Lock for shared resources
 -  Pro: True parallelism on multi-core
 -  Con: Complexity, potential deadlocks
- **Queue-based processing:** Use queue.Queue (thread-safe by default)
 -  Pro: Built-in thread safety, simple
 -  Con: Still need locks for helper states
- **Async/await:** Use asyncio for concurrent operations
 -  Pro: Efficient for I/O-bound operations
 -  Con: More complex for beginners

My Recommendation: Threading with Queue + Locks - Good balance of safety and performance. Python's `queue.Queue` is thread-safe, and we'll use locks for helper availability.

4. Privacy & Data Handling

Strategy:

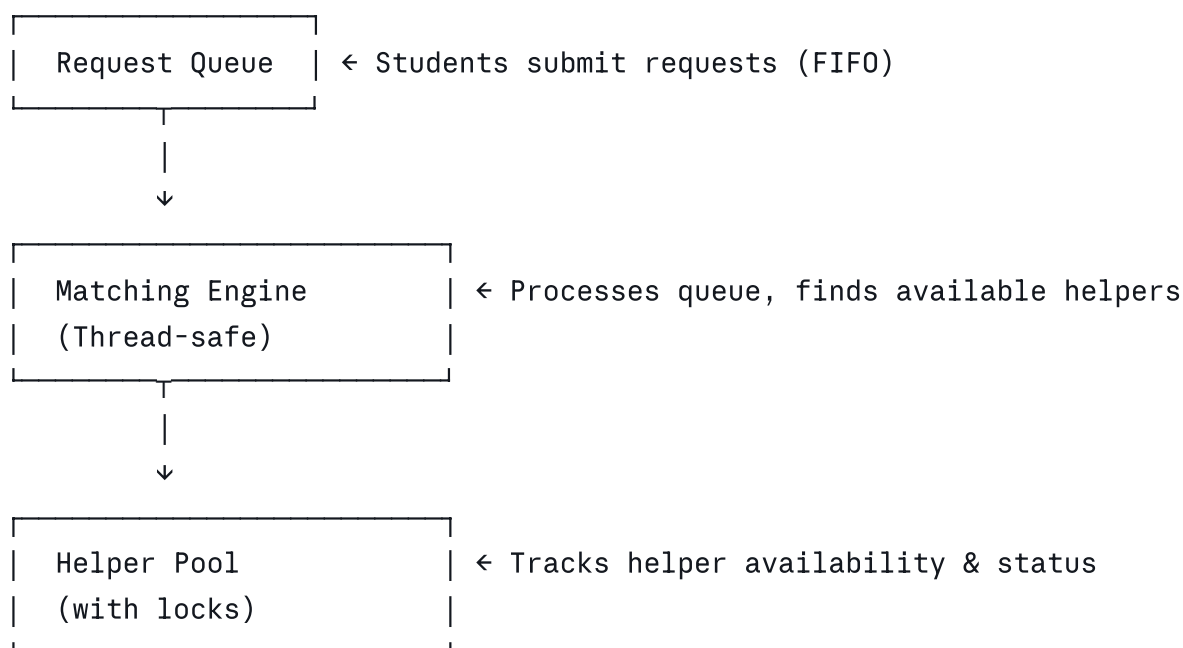
- Generate anonymous IDs for students in requests

- Share only: pickup location, destination, time (not student name/details)
 - Reveal identities only after match confirmation
 - Implement secure messaging channel
-

5. Edge Case Handling

- **No helpers available:** Queue request until helper becomes available
 - **Multiple helpers respond:** Use locking - first to accept wins
 - **Helper doesn't show up:** Timeout mechanism, requeue request
 - **Student cancels:** Remove from queue, notify assigned helper
 - **Helper becomes unavailable:** Remove from pool, reassign their pending requests
-

Proposed Architecture



What do you think of this approach?

Specifically:

1. **Queue strategy:** FIFO fair, or should we add priority for urgent/late-night requests?
2. **Assignment:** Hybrid offer-accept, or full auto-assignment for speed?
3. **What timeout values** seem reasonable (e.g., helper has 30 seconds to respond)?

Let me know your preferences and I'll build the system accordingly! 🚀

i am satisfied with the system overview and the decisions

1. u think just do FIFO
2. hybrid offer accept
3. 30 seconds sounds good

now can u pls provide me the python file

Creating the Safe Walk companion matching system with queue-based request management, hybr... >

Now let me create the CHANGES.docx file documenting the requirements and implementation:

Ran 2 commands, viewed a file, created a file >

Perfect! Now let me copy the Python file to the outputs directory and share both files with you:

Ran a command, read a file >

Perfect! I've created both files for you:

Files Created

1. safe_walk_system.py - The complete Safe Walk Companion System with:

-  **FIFO Queue System** for fair request processing
-  **Hybrid Offer-Accept** model preventing helper conflicts
-  **Thread-safe operations** using locks and queues
-  **30-second timeout** with automatic requeuing
-  **Privacy protection** via anonymous IDs
-  **Secure messaging** (only after match confirmation)
-  **Comprehensive edge case handling**
-  **Extensive comments** for Python learners
-  **Complete demo** showing all scenarios

2. CHANGES.docx - Full requirements documentation including:

- All 8 functional requirements mapped to implementation
- Non-functional requirements (responsiveness, reliability)
- Design decisions with rationale

- Edge cases handled
- Error handling patterns
- Code organization guide

System Architecture Overview

The system uses a **FIFO queue** because it's fair and simple - first student to request gets priority. The **hybrid offer-accept** model was chosen because it prevents conflicts (only one helper considers a request at a time) while respecting helper autonomy (they can decline).

Key Classes:

- **Student** - Requests with privacy (anonymous IDs)
- **Helper** - Volunteers with status tracking
- **WalkRequest** - Walk requests with status transitions
- **SafeWalkSystem** - Main coordinator with thread-safe operations
- **MessagingSystem** - Secure post-match communication

You can run the demo by executing: `python safe_walk_system.py`

